

Chapter 1: Introduction

1.1 Overview

Campus AI Hub is a comprehensive, AI-powered campus management and learning platform designed to modernize the academic ecosystem of higher education institutions. Despite modern tech, plenty of campuses stick with paper trails alongside scattered software bits. Because these systems do not talk to one another, tasks slow down, messages get lost, information piles up in duplicate, while learners miss tailored guidance they could benefit from.

One way the Campus AI Hub helps? It brings students, teachers, and staff together online using one smart system. Instead of just managing courses like older platforms, it weaves artificial intelligence into everyday school tasks. Think guided studying powered by AI - something most learning systems don't do. Another feature: practice interviews shaped by machine feedback. Not only does it streamline access, it reshapes how support works across campus.

Built to work smoothly on any screen size, from phones to big monitors. What drives it? A smarter way to run campus life - linking tools so students learn better, staff get more done. No matter where you log in, things just fit, right away. Thinking ahead keeps the design clean: one system that grows with real needs. Every piece connects quietly, behind the scenes, making daily tasks lighter. Outcomes improve when tech fades into the background, helpful but unseen. Speed meets simplicity, especially during busy class changes or report seasons. Access stays consistent, whether sitting at a desk or walking between buildings. Effort drops when everything speaks the same digital language. Even complex schedules feel manageable because the interface adapts. Learning flows easier when friction disappears from routine actions. Behind every click, there's planning focused solely on usefulness.

1.2 Project Description

Inside the Campus AI Hub, everything ties together online for school stuff. Depending on who you are, the system shows different options and info. Logging in opens paths based on your role - student, teacher, or manager. Learners look up class files anytime, turn in homework, try out smart helpers, then watch how they improve over time. Teachers run their classes right from here - posting guides, making tasks, grading work, staying in touch with those enrolled. Those running things behind the scenes handle accounts, organize subjects, keep an eye on usage trends. Access shifts quietly depending on responsibility level built into each profile. What appears on screen fits exactly what one needs to do next.

Instead of just teaching, Campus AI Hub builds smarts into learning. One part answers questions, boils down lessons, gives clean summaries. Another runs mock tech chats where replies get checked on the spot. Feedback comes quick, clear, shows where to grow. Outcomes shape learners who handle class pressure plus job demands alike.

1.2 Objectives of the Project

The primary objectives of the Campus AI Hub project are:

- To digitize and centralize campus academic and administrative processes.
- To reduce dependency on multiple disjointed tools and manual systems.
- To provide personalized AI-driven academic support to students.
- To enhance communication between students, faculty, and administrators.
- To build a scalable, secure, and user-friendly web application using modern technologies.

1.3 Scope of the Project

The scope of the project includes:

- Development of a full-stack web application with role-based access.
- Implementation of course management, assignment handling, and timetable features.
- Integration of AI modules for study assistance and interview practice.
- Secure authentication and authorization mechanisms.
- Deployment-ready architecture suitable for real-world institutional use.

The project does not include physical infrastructure management or financial management modules such as fee payment systems, which can be considered for future extensions.

1.4 Company / Institution Profile

This is committed to fostering academic excellence and technological innovation. Within the framework of its curriculum, students are motivated to undertake capstone projects that translate theoretical knowledge into practical solutions. Campus AI Hub is developed as a capstone project to demonstrate practical skills in full-stack development, database design, system analysis, and AI integration.

Chapter 2: Literature Survey

2.1 Introduction to Literature Survey

A literature survey involves the systematic study of existing systems, tools, and methodologies related to the problem domain. For Campus AI Hub, the survey focuses on traditional campus management systems, existing Learning Management Systems (LMS), and recent advancements in AI-driven educational platforms. The goal is to identify limitations in current solutions and justify the require for the proposed system.

2.2 Existing platform

The existing academic environment, most institutions rely on a combination of manual and semi-digital processes:

- **Manual Notice Boards:** Important announcements, timetables, and circulars are physically displayed, leading to delays and missed information.
- **Disjointed Digital Tools:** Communication through messaging apps, assignments through LMS tools, and results via separate portals create confusion.
- **Lack of Personalization:** Students receive generic learning resources without personalized guidance or interview preparation support.
- **Data Redundancy:** Student and course data are repeatedly entered across multiple systems, increasing the risk of errors.

These limitations negatively impact operational efficiency and student learning outcomes.

2.3 Proposed System

Campus AI Hub proposes a unified and intelligent alternative:

- **Centralized Dashboard:** A single platform for all academic activities.
- **AI Integration:** Real-time doubt clarification, note generation, and interview practice.
- **Role-Based Access:** Secure and customized interfaces for Students, Faculty, and Admins.
- **Real-Time Updates:** Instant access to notices, assignments, grades, and schedules.

By consolidating all features into one system, Campus AI Hub eliminates redundancy and improves the overall academic experience.

2.4 Feasibility Study

2.4.1 Technical Feasibility

The project uses modern, open-source technologies such as React, Node.js, and Prisma. These technologies are well-documented, widely supported, and capable of supporting scalable web applications.

Operational Feasibility

The system is designed with a user-friendly interface, ensuring minimal learning curve. Students and faculty can easily adapt to the platform without extensive training.

2.4.2 Economic Feasibility

Since the project relies on free and open-source software, development costs are minimal. The only recurring expense is hosting, making it cost-effective for educational institutions.

Chapter 3: Software Requirements Specification (SRS)

3.1 Introduction to SRS

The Software Requirements Specification (SRS) document defines the complete functional and non-functional requirements of the Campus AI Hub system. It serves as a formal agreement between stakeholders and developers, clearly describing what the system is expected to do. This chapter acts as a foundation for system design, implementation, and testing.

The SRS ensures clarity, reduces ambiguity, and helps in validating whether the final result meets user needs and institutional needs.

3.2 Purpose of the System

The primary purpose of Campus AI Hub is to provide a unified digital platform that supports academic activities, communication, and AI-assisted learning within a campus. The system aims to:

- Digitize academic workflows.
- Improve accessibility to learning resources.
- Provide personalized AI-based academic and career support.
- Enable efficient monitoring and management by administrators.

3.3 Scope of the System

Campus AI Hub covers the following functional scope:

- User authentication and role-based authorization.
- Course and module management.
- Uploading and accessing notes, videos, and assignments.
- AI-powered study assistant and interview preparation.
- Academic scheduling and timetable management.
- Community-based discussion forums.

The platform is scalable and can be extended with additional modules such as mobile applications, proctoring systems, and advanced analytics.

3.4 User Classes and Characteristics

3.4.1 Student

Students are the primary users of the platform. They are expected to have basic knowledge of using web applications.

Responsibilities: - Access course materials and video lectures. - Submit assignments before deadlines. - Interact with the AI Study Assistant. - Participate in AI Interview Practice sessions. - View timetables, notices, and grades.

3.4.2 Faculty

Faculty members manage academic content and student evaluations.

Responsibilities: - Create and manage courses and modules. - Upload notes and video lectures. - Create assignments and grade submissions. - Provide feedback to students.

3.4.3 Administrator

Administrators manage the overall system.

Responsibilities: - User management (add/remove/update users). - Course oversight and allocation. - Monitor platform usage and analytics. - Ensure data integrity and system availability.

3.5 Operating Environment

- Client Side: Modern web browsers (Chrome, Edge, Firefox, Safari).
- Server Side: Node.js runtime environment.
- Database: SQLite (development), PostgreSQL (production).
- Hosting: Cloud-based server or institutional data center.

3.6 Functional Requirements

3.6.1 Authentication and Authorization

- The system shall allow users to register and log in securely.
- The system shall verify user roles (student, faculty, admin).
- Unauthorized users shall be denied access to protected resources.

3.6.2 Course Management

- Faculty shall be able to create and update courses.
- Courses shall be divided into multiple modules.
- Students shall be able to enroll and view assigned courses.

3.6.3 Assignment Management

- Faculty shall create assignments with deadlines.
- Students shall upload submissions before the deadline.
- Faculty shall evaluate and grade submissions.

3.C.4 AI Assistant Module

- The system shall provide an AI chat interface.
- Students shall ask academic questions and receive responses.
- The interview mode shall simulate technical interview scenarios.

3.C.5 Community and Communication

- Users shall participate in discussion forums.
- Students shall post questions and receive peer/faculty responses.

3.7 Non-Functional Requirements

3.7.1 Performance Requirements

- Pages should load within 2 seconds under normal conditions.
- AI responses should be generated within acceptable latency (<3 seconds).

3.7.2 Security Requirements

- Passwords shall be stored in encrypted format.
- JWT-based authentication shall secure API endpoints.
- Role-based access control shall be enforced.

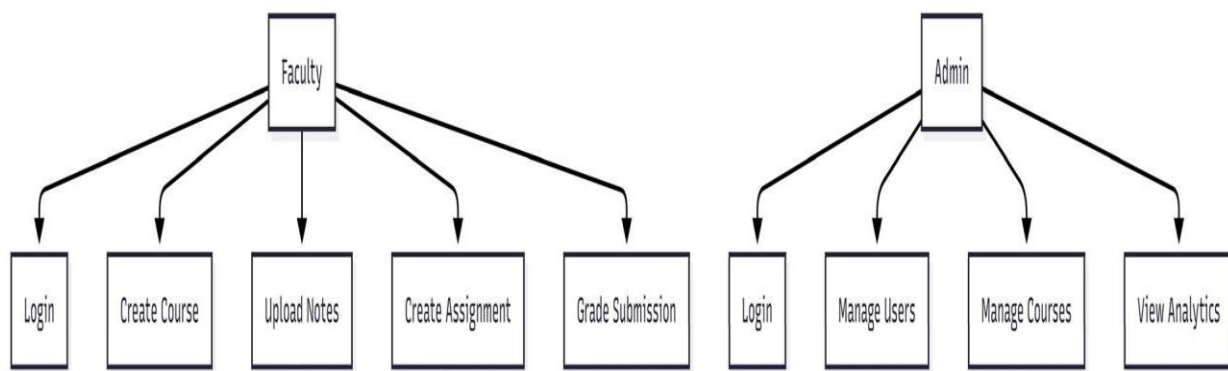
3.7.3 Reliability and Availability

- The system should maintain 99.9% uptime during academic hours.
- Backup mechanisms should prevent data loss.

3.7.4 Scalability

- The system shall support concurrent users without performance degradation.

3.8 System Use Case Diagram (Mermaid)



3.9 Assumptions and Dependencies

- Users have reliable internet access.
- AI services are available and responsive.
- Users possess basic digital literacy.

3.10 Constraints

- System functionality depends on third-party AI APIs.
- Performance may vary based on hosting infrastructure.

Chapter 4: System Design

4.1 Introduction to System Design

Platform design tells how the requirements specified in the SRS are translated into a structured and workable architecture. This chapter explains the overall design approach of Campus AI Hub, including architectural style, system components, data flow, and interactions between users and external services.

The design focuses on scalability, modularity, security, and maintainability, ensuring that the system can support real-world academic environments with multiple concurrent users.

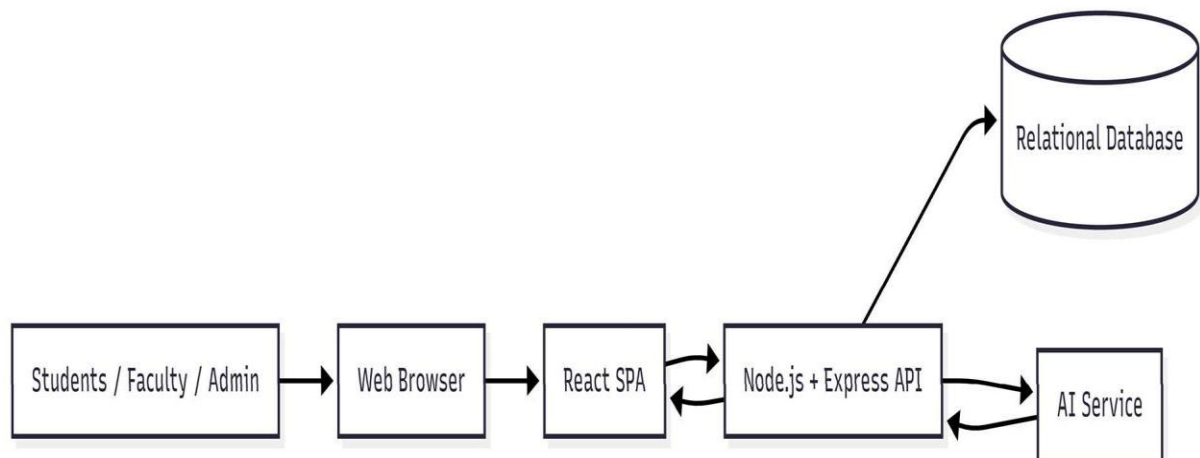
4.2 System Architecture Overview

Campus AI Hub follows a **Client–Server Architecture** combined with a **Single Page Application (SPA)** model on the frontend. The architecture is divided into 3 main layers:

1. Presentation Layer (Frontend)
2. Application Layer (Backend)
3. Data Layer (Database)

Each layer is independent yet interconnected, allowing easier maintenance and future enhancements.

4.3 High-Level Architecture Diagram



Explanation: - Users interact with the system through a web browser. - The React-based frontend handles UI rendering and user interaction. - The backend API processes business logic, authentication, and validation. - The database stores persistent academic data. - The AI service provides intelligent responses for study assistance and interview practice.

4.4 System Components Description

4.4.1 Frontend Layer

The frontend is developed using React with TypeScript and Tailwind CSS. It is responsible for:

- Rendering dashboards based on user roles.
- Handling client-side routing.
- Communicating with backend APIs using HTTP requests.
- Managing application state and form validations.

4.4.2 Backend Layer

The backend is built using Node.js and Express.js. It performs:

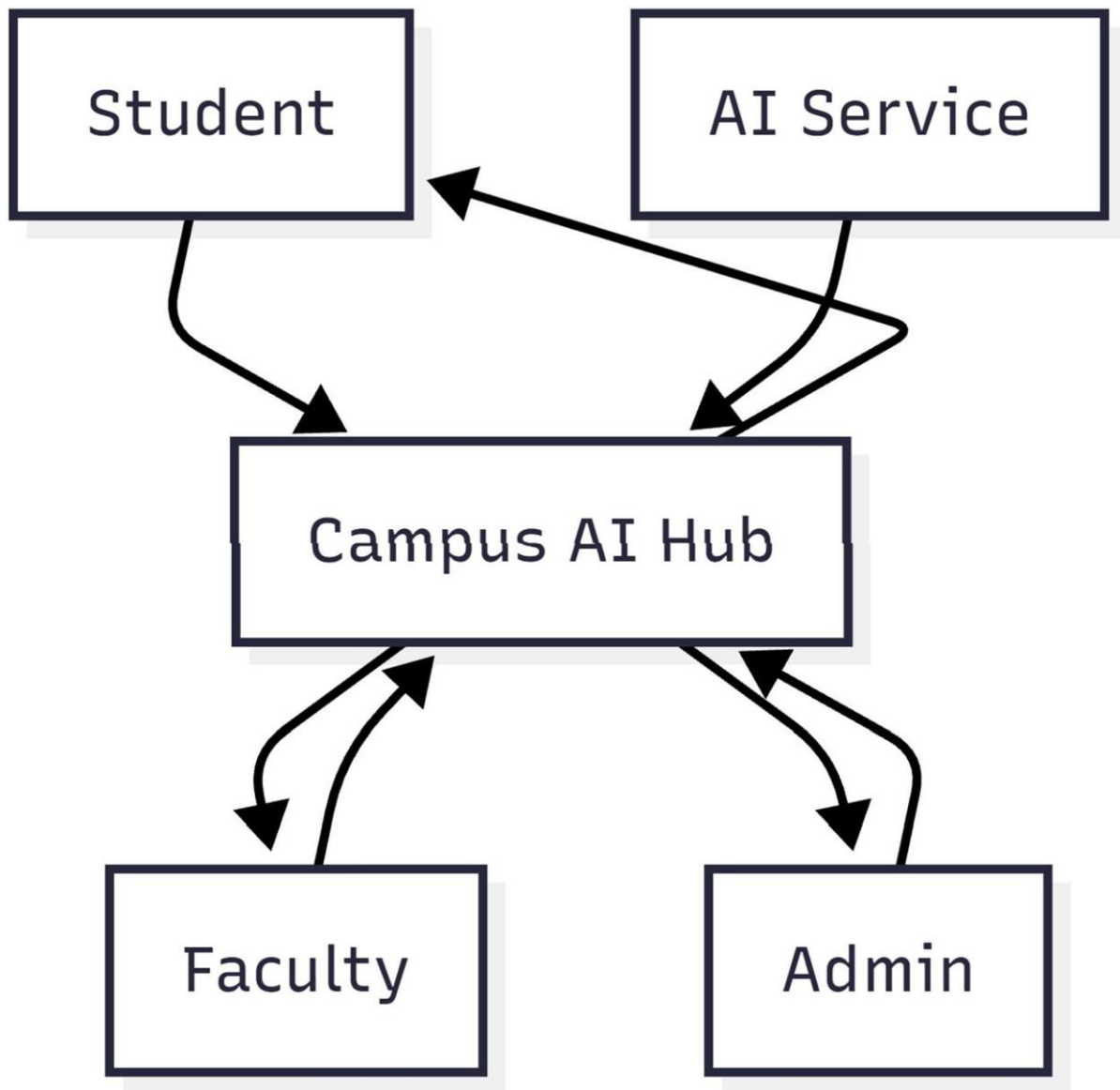
- Authentication and authorization using JWT.
- Business logic execution.
- Validation of user inputs.
- Communication with AI services.
- Database operations via Prisma ORM.

4.4.3 Database Layer

A relational database is used to store structured data such as users, courses, assignments, and submissions. Relationships between entities ensure data consistency and integrity.

4.5 Context Diagram

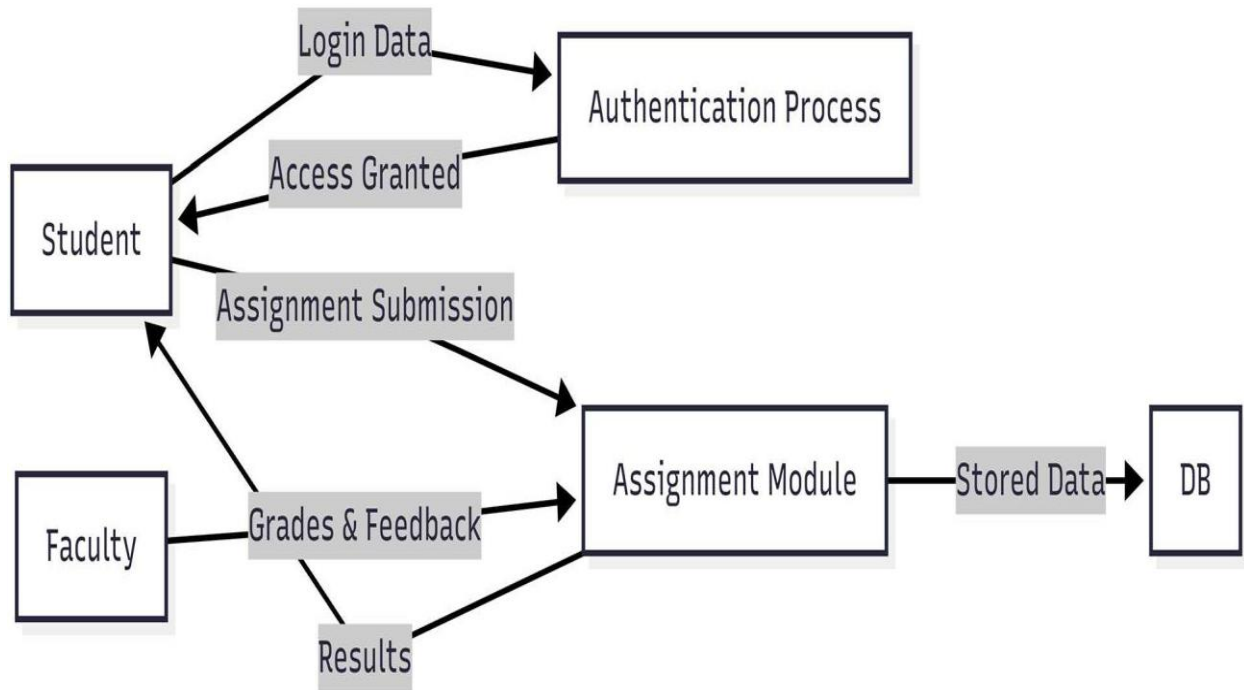
The context diagram represents the system as a single process and shows interactions with external entities.



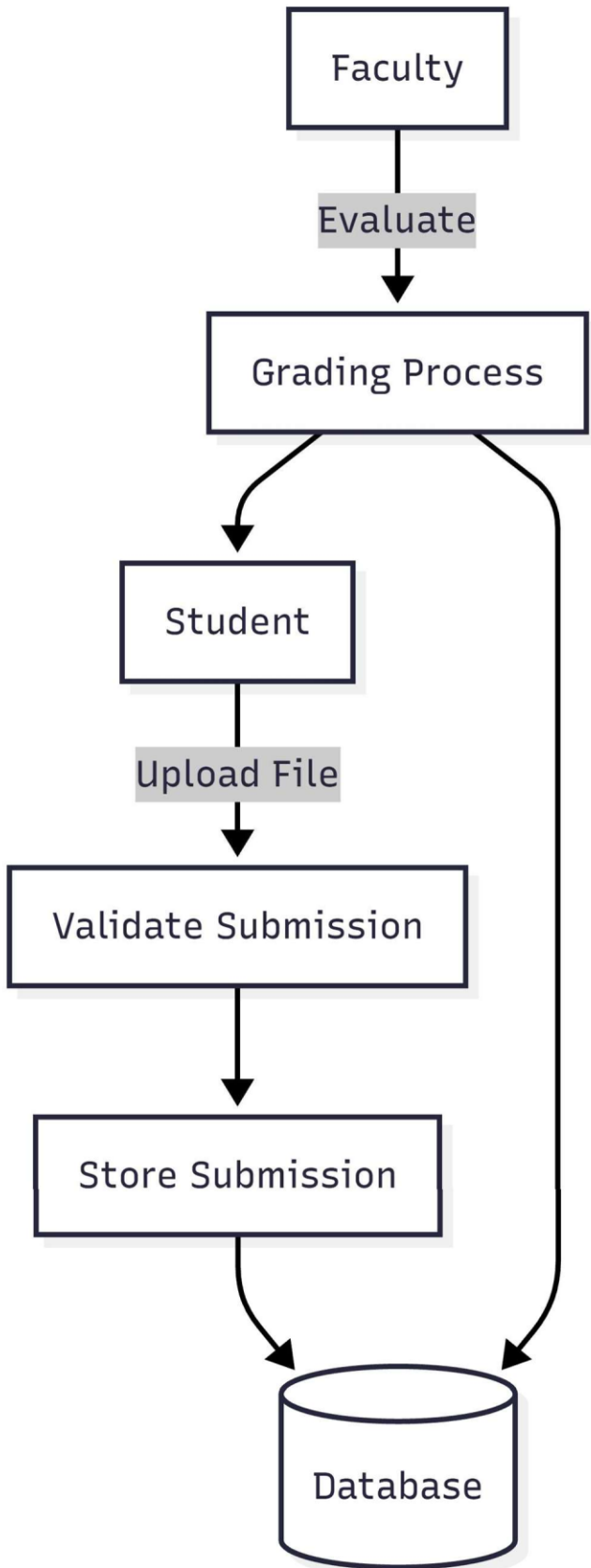
Explanation: - Students, Faculty, and Admins interact directly with Campus AI Hub. - The system communicates with external AI services to generate responses and feedback.

4.6 Data Flow Diagram (DFD – Level 0)

The Level 0 DFD illustrates the overall flow of data within the system.



4.7 Data Flow Diagram (DFD – Level 1: Assignment Module)



4.8 Deployment Diagram

The deployment diagram describes how software components are deployed on hardware nodes.



4.9 Design Considerations

- **Scalability:** Modular architecture supports future expansion.
- **Security:** JWT-based authentication and encrypted passwords.
- **Performance:** Optimized APIs and efficient database queries.
- **Maintainability:** Separation of concerns ensures easier debugging and upgrades.

4.10 Summary

This chapter presented the architectural and design aspects of Campus AI Hub. The layered architecture, clear data flow, and modular components ensure that the system is robust, scalable, and suitable for deployment in real academic institutions.

Chapter 5: Detailed Design

5.1 Introduction to Detailed Design

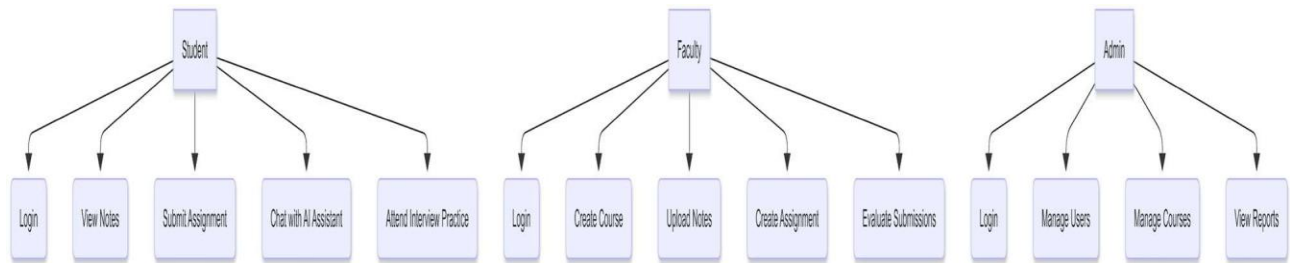
Detailed design translates the system architecture into precise, implementable components. This topic describes how each module of Campus AI Hub functions internally, how users interact with the system, and how data flows between components. The design emphasizes clarity, modularity, and maintainability.

5.2 Use Case Design

Use case design explains system functionality from the user's perspective. Each use case represents a specific interaction between an actor and the system.

5.2.1 Use Case Diagram (Overview)

5.3 Use Case Descriptions



Use Case 1: User Login

- **Actor:** Student / Faculty / Admin
- **Description:** Allows users to authenticate and access role-based dashboards.
- **Precondition:** User must be registered.
- **Postcondition:** User session is created.

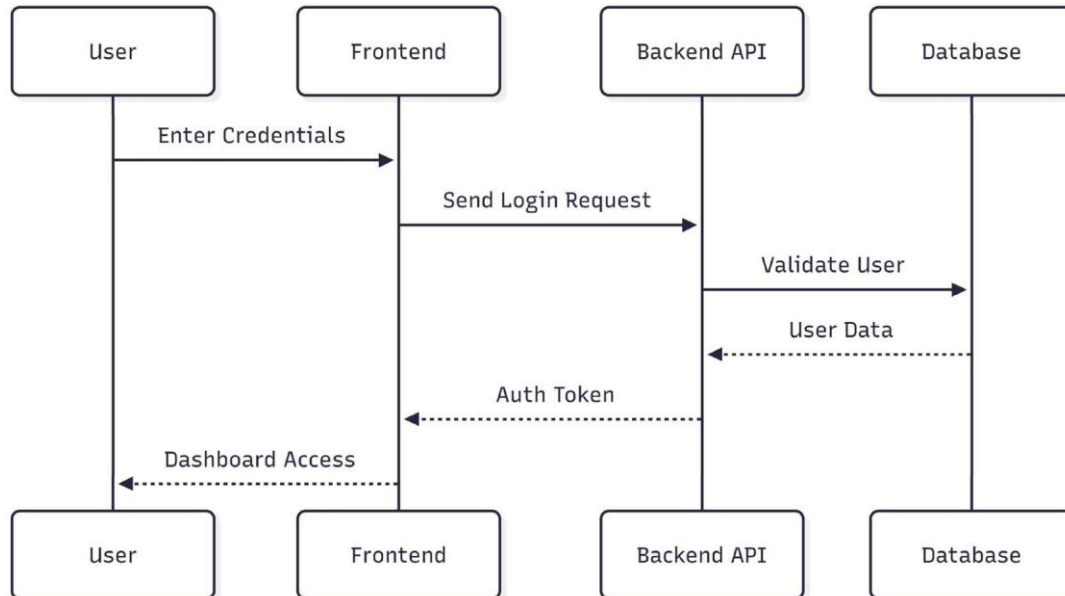
Use Case 2: Submit Assignment

- **Actor:** Student
- **Description:** Enables students to upload assignment files before deadlines.
- **Precondition:** Assignment must be active.
- **Postcondition:** Submission stored in database.

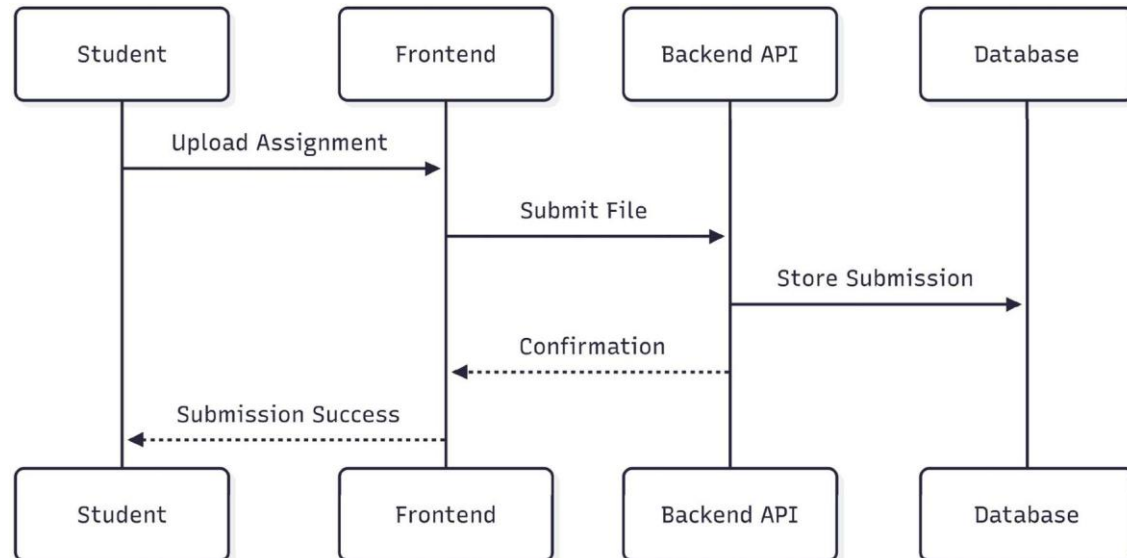
Use Case 3: Grade Assignment

- **Actor:** Faculty
- **Description:** Faculty evaluates submissions and assigns grades.
- **Precondition:** Submission exists.
- **Postcondition:** Grade visible to student.

5.4 Sequence Diagram – Login Process



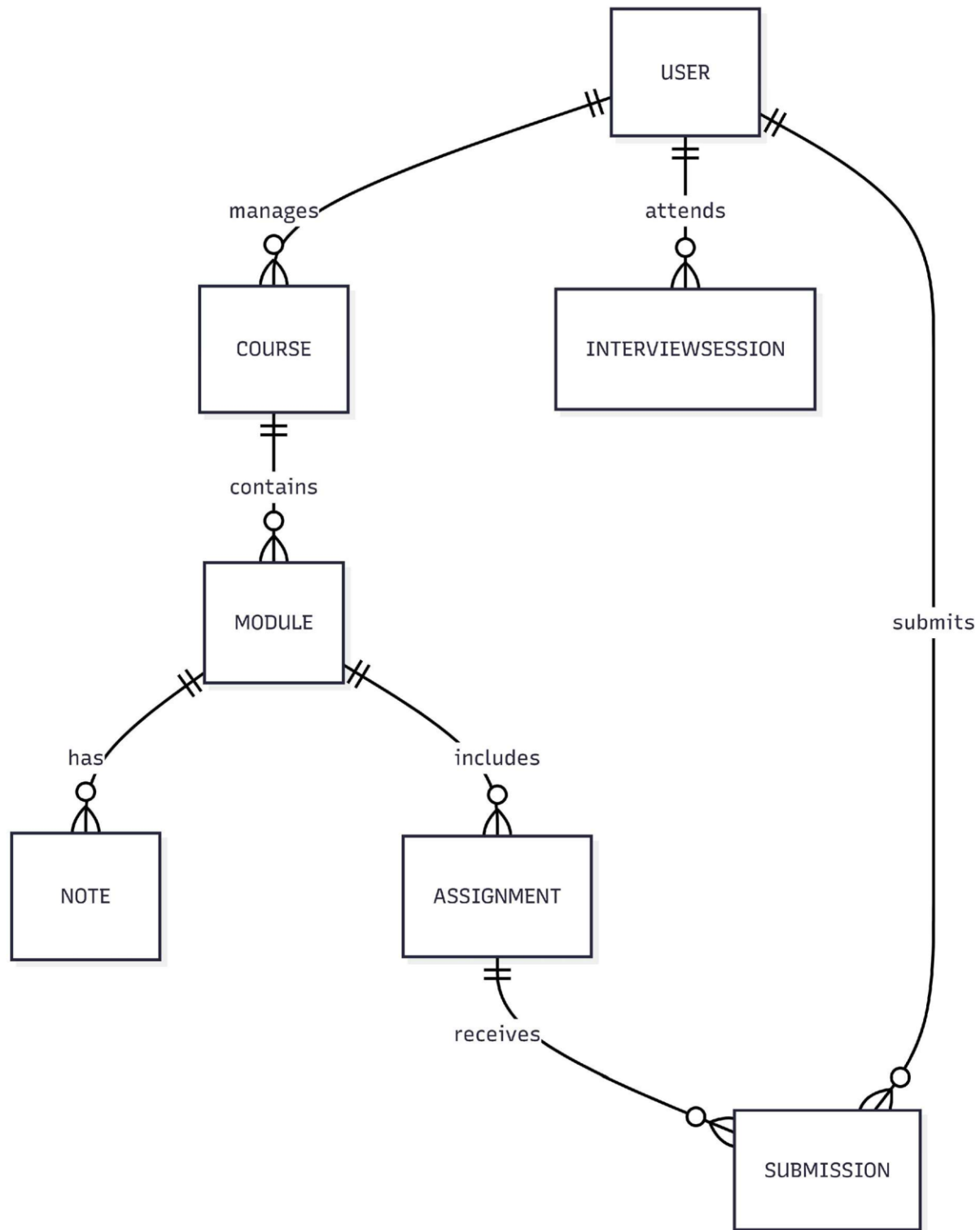
5.5 Sequence Diagram – Assignment Submission



5.6 Database Design

The database design follows a relational model ensuring data consistency and integrity. Prisma ORM is used for schema definition and database interaction.

5.C.1 Entity Relationship Diagram (ER Diagram)



5.7 Database Tables Description

- **User:** Stores authentication and role information.
- **Course:** Represents academic courses.
- **Module:** Logical units within a course.
- **Assignment:** Tasks assigned to students.
- **Submission:** Student submissions and grades.
- **InterviewSession:** Stores AI interview feedback.

5.8 Module-Level Design

5.8.1 Authentication Module

- Handles login, signup, and token validation.
- Uses JWT for secure session management.

5.8.2 Course Management Module

- Enables faculty to create and manage courses.
- Controls student enrollment.

5.8.3 AI Assistant Module

- Processes user queries.
- Communicates with AI APIs.
- Stores interaction history.

5.9 Design Constraints

- Dependency on external AI services.
- Network latency may affect AI response time.

5.10 Summary

This chapter presented the detailed internal design of Campus AI Hub, including use cases, sequence diagrams, ER diagram, and module-level design. These designs provide a clear blueprint for implementation.

Chapter 6: Implementation

Chapter 6: Screenshots of the Project

6.1 Introduction

This chapter presents the visual representation of the Campus AI Hub application through screenshots. Screenshots provide a clear understanding of the user interface, navigation flow, and major functionalities implemented

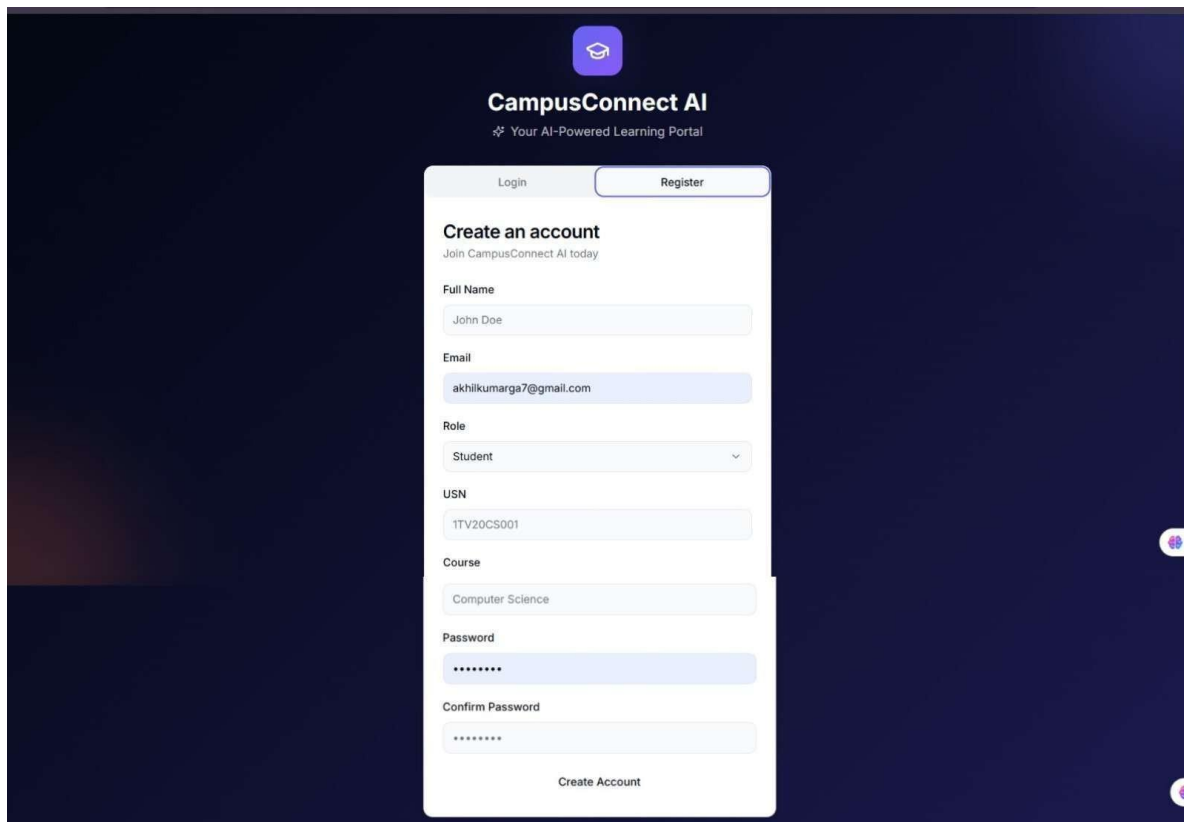
in the system. They help evaluators and readers visualize how the system operates in a real-world scenario.

The screenshots included in this chapter demonstrate role-based access, core academic workflows, and AI-powered features of the platform.

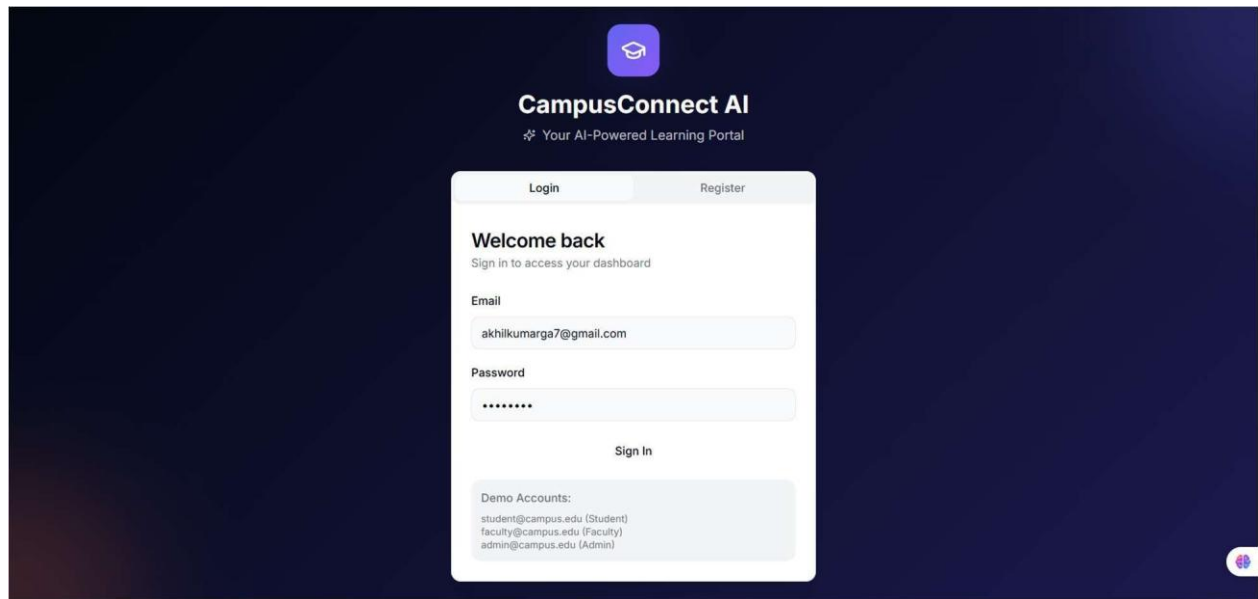
6.2 Login Page

Description: The Login Page is the entry point of the Campus AI Hub system. It allows registered users to authenticate themselves using their email ID and password. Role-based redirection ensures that students, faculty, and administrators are directed to their respective dashboards upon successful login.

Screenshot Placeholder:



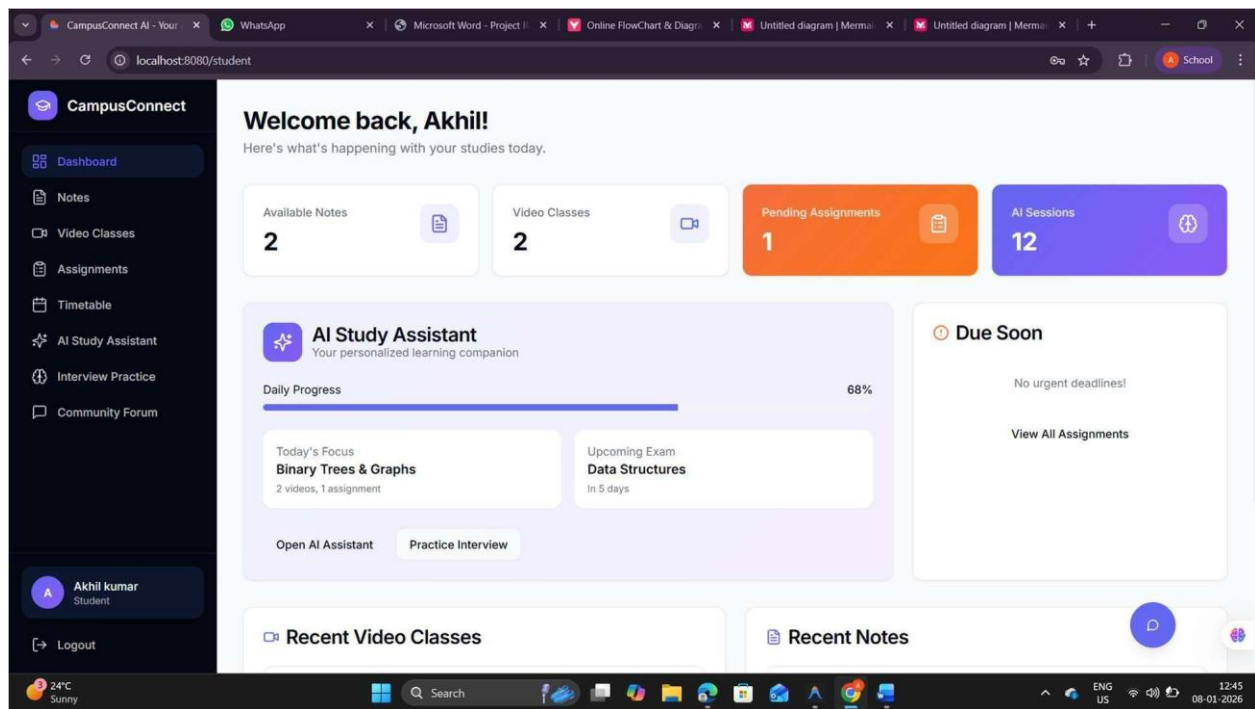
The screenshot displays the 'CampusConnect AI' login and registration interface. At the top, a purple graduation cap icon is positioned above the text 'CampusConnect AI' and the tagline 'Your AI-Powered Learning Portal'. Below this, there are two tabs: 'Login' and 'Register'. The 'Register' tab is currently selected, showing a 'Create an account' form. The form includes fields for 'Full Name' (with the placeholder 'John Doe'), 'Email' (with the placeholder 'akhiikumarga7@gmail.com'), 'Role' (a dropdown menu set to 'Student'), 'USN' (with the placeholder '1TV20CS001'), 'Course' (with the placeholder 'Computer Science'), 'Password', and 'Confirm Password'. All input fields are light blue. At the bottom of the form is a 'Create Account' button. The entire form is centered on a dark blue background.



6.3 Student Dashboard

Description: The Student Dashboard provides an overview of enrolled courses, upcoming assignments, notices, and access to the AI Assistant. It acts as a centralized control panel for students to manage their academic activities efficiently.

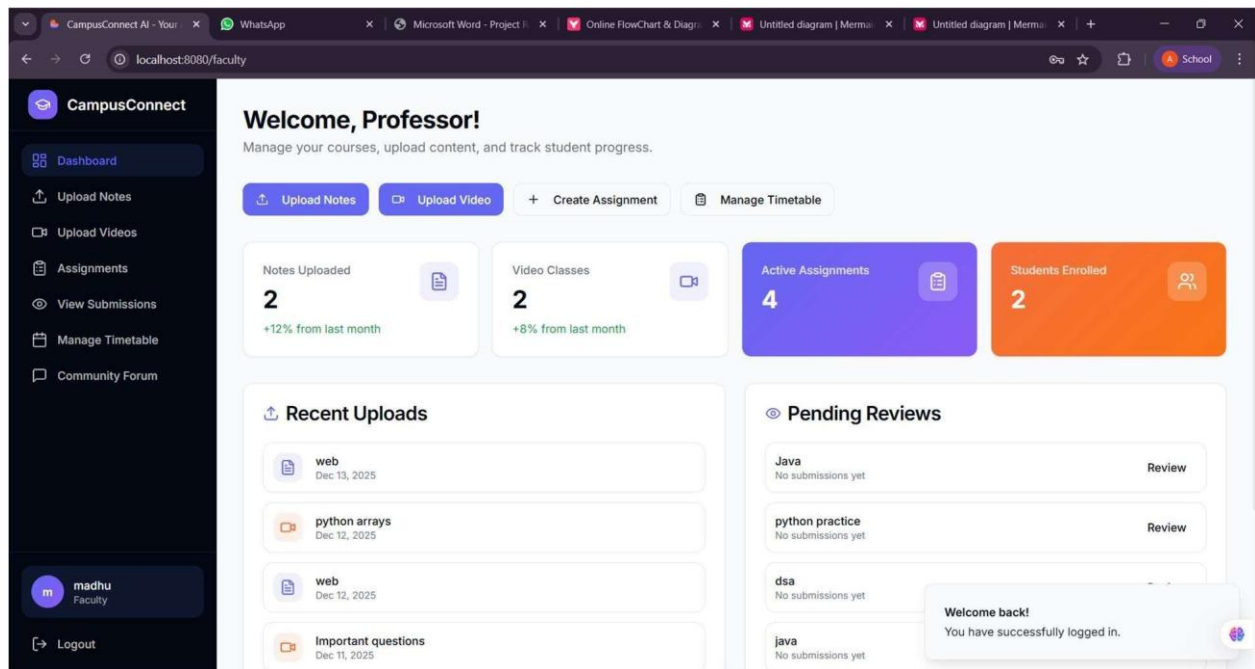
Screenshot Placeholder:



6.4 Faculty Dashboard

Description: The Faculty Dashboard enables faculty members to manage courses, upload notes and videos, create assignments, and evaluate student submissions. The interface is designed to simplify academic content management.

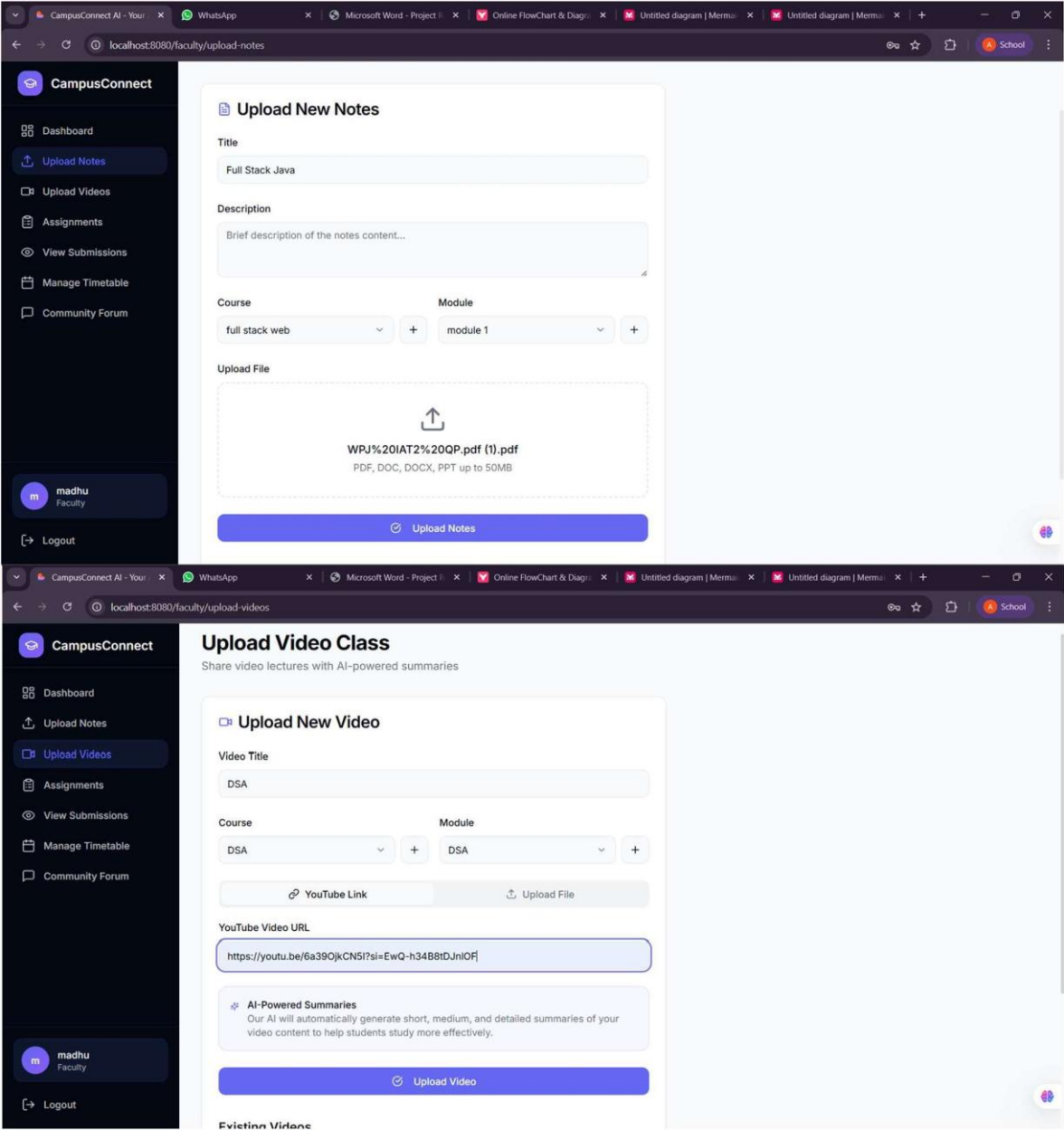
Screenshot Placeholder:

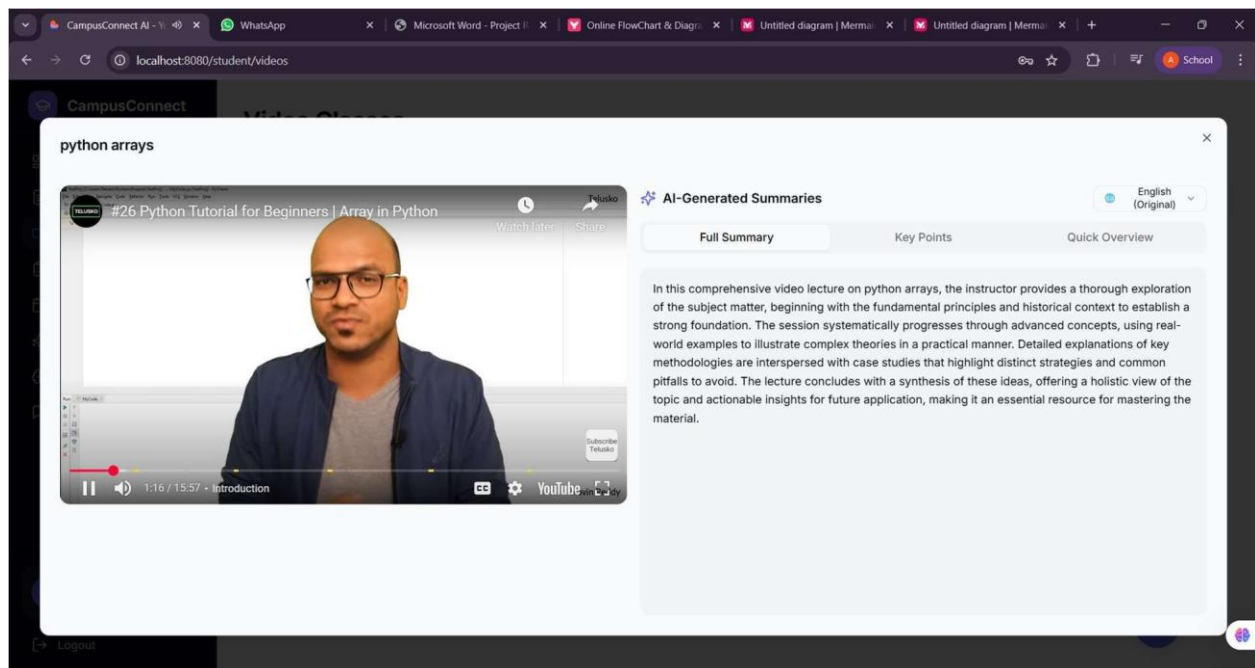


6.5 Course and Module Management

Description: This screen displays the list of courses and their respective modules. Faculty can add, update, or remove modules, while students can access notes, videos, and assignments associated with each module.

Screenshot Placeholder:

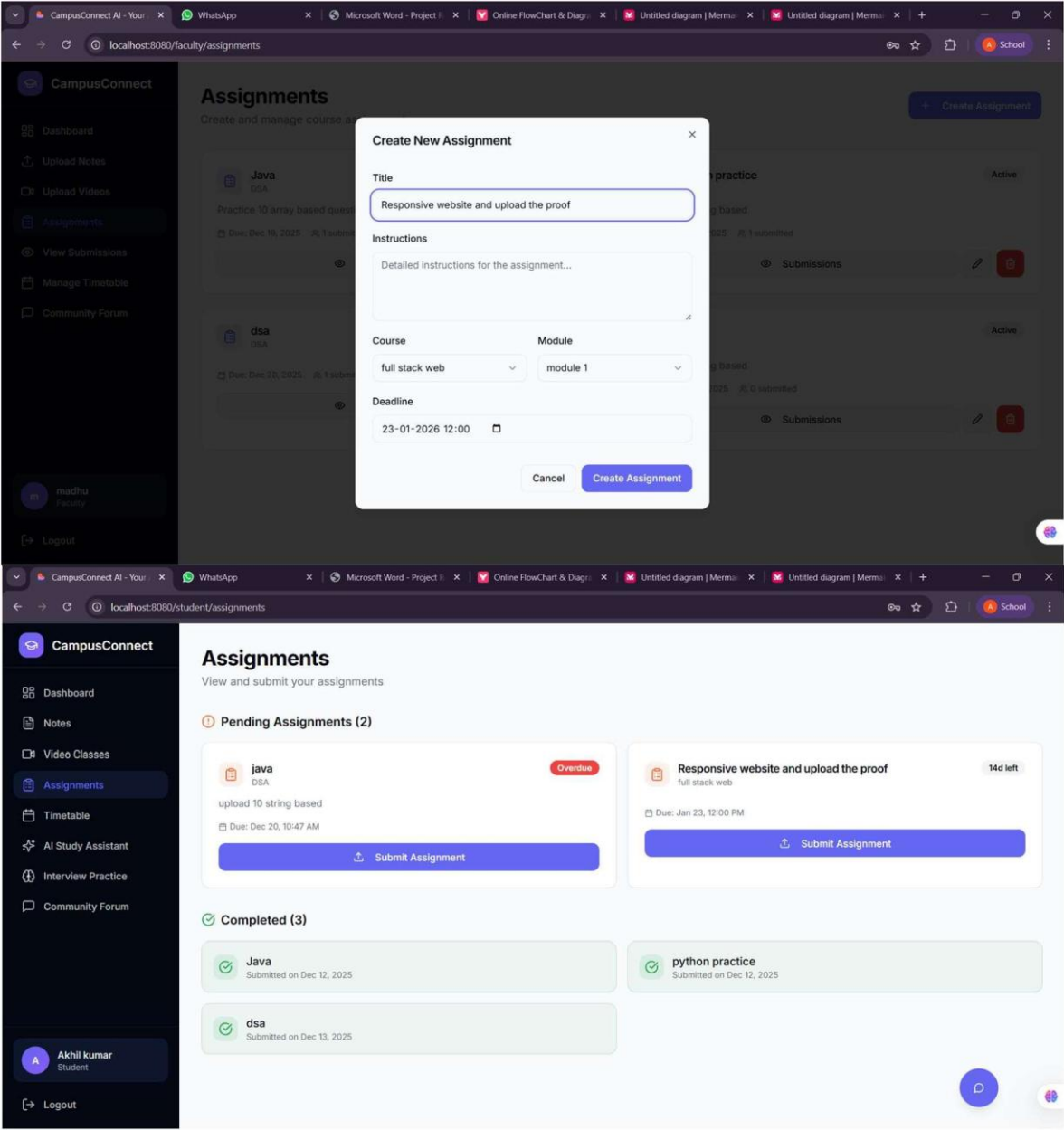


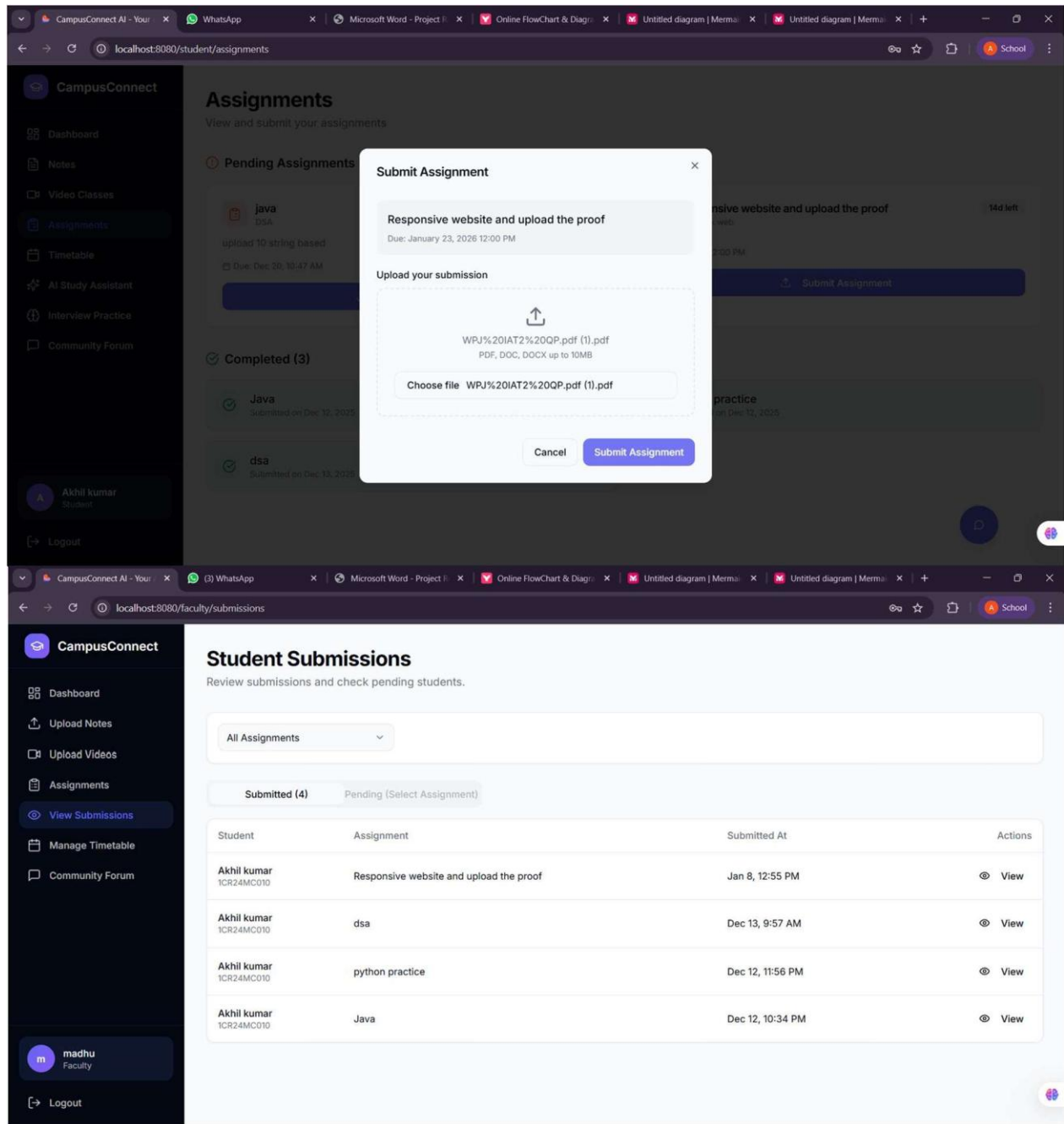


6.6 Assignment Creation and Submission

Description: This screen shows the assignment workflow. Faculty can create assignments with deadlines, and students can upload their submissions. The system automatically enforces deadlines and confirms successful submission.

Screenshot Placeholder:

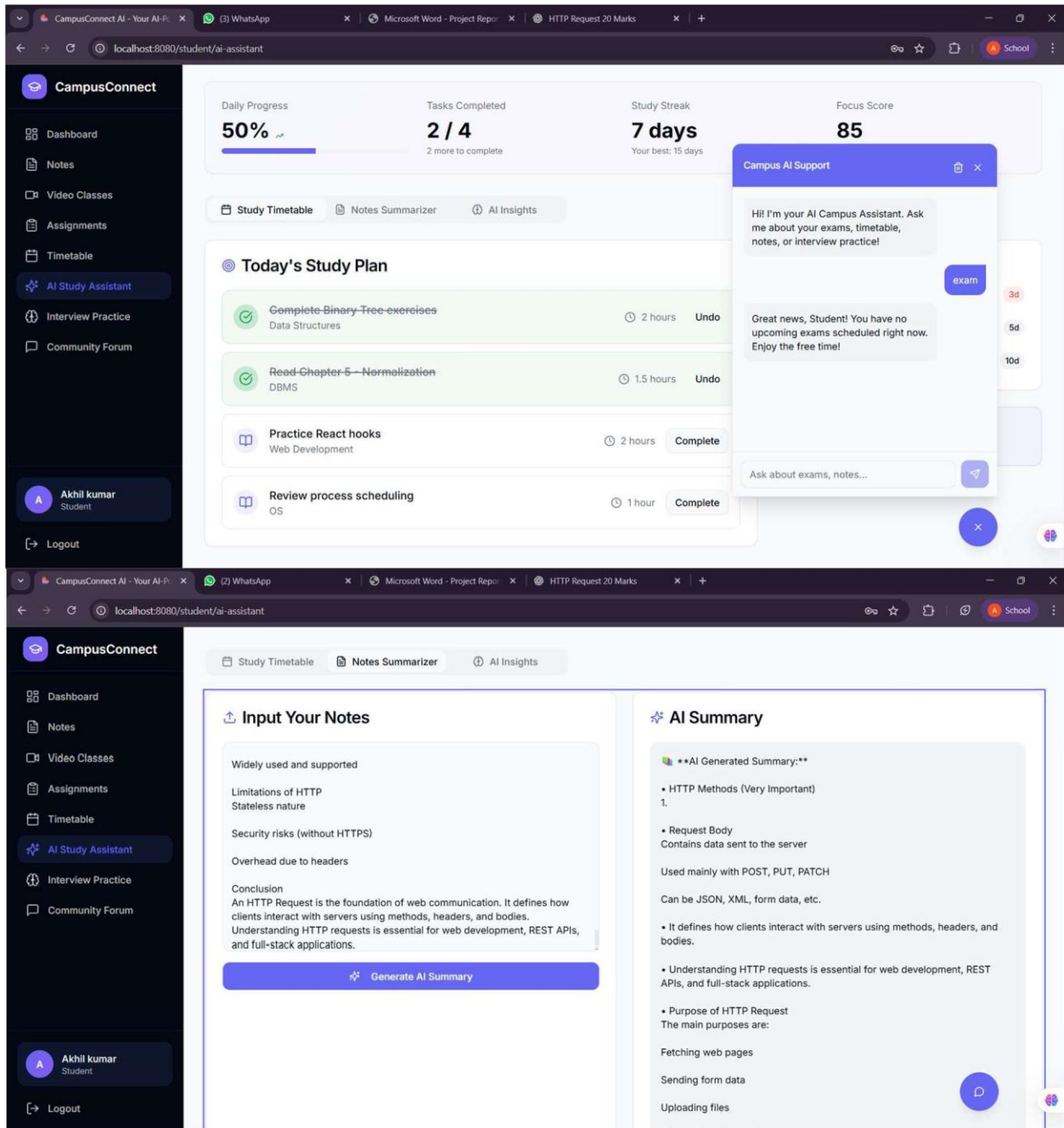




6.7 AI Study Assistant

Description: The AI Study Assistant screen provides a chat-based interface where students can ask academic questions, request explanations, and generate summarized notes. Responses are generated in real time using AI services.

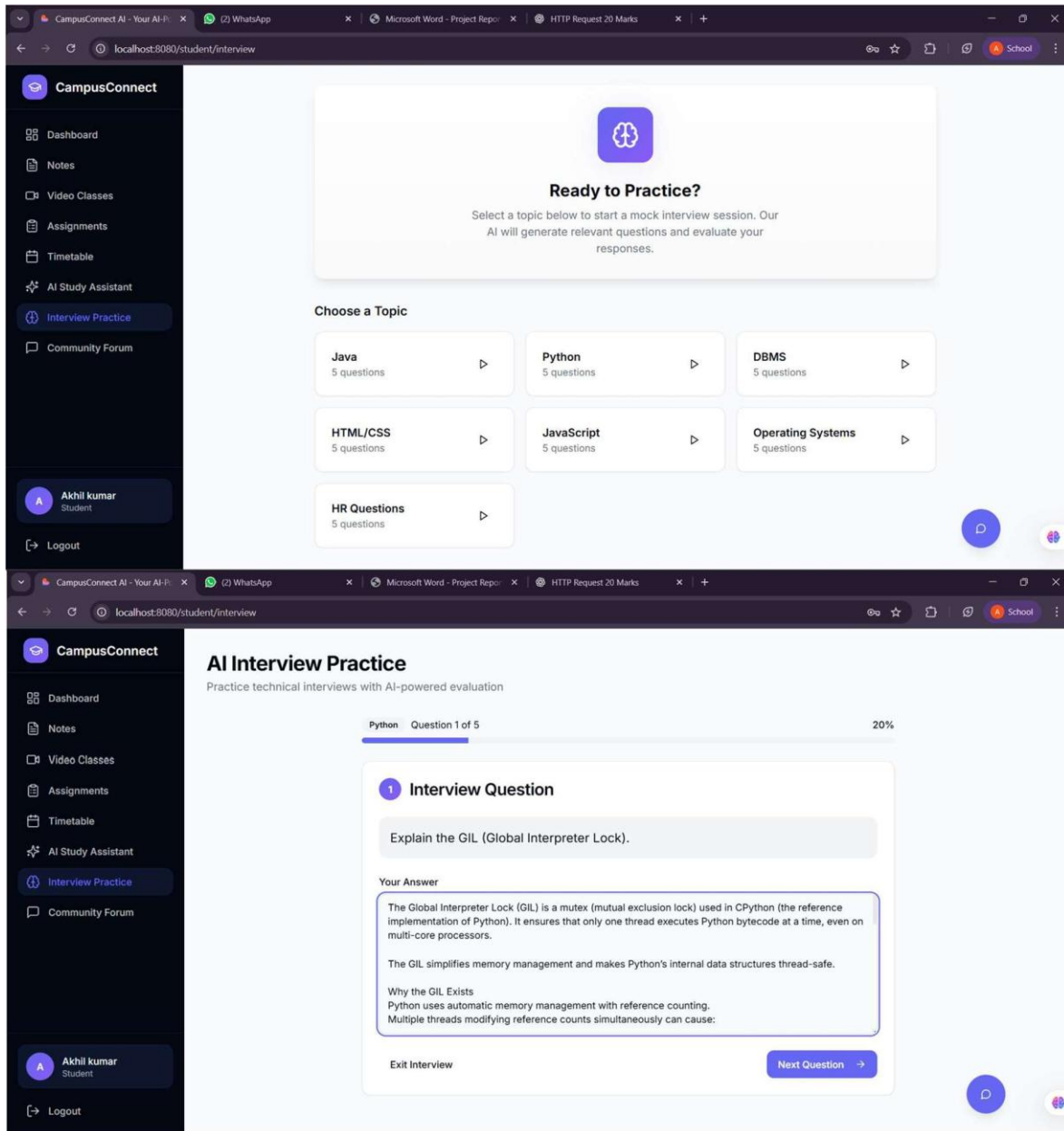
Screenshot Placeholder:

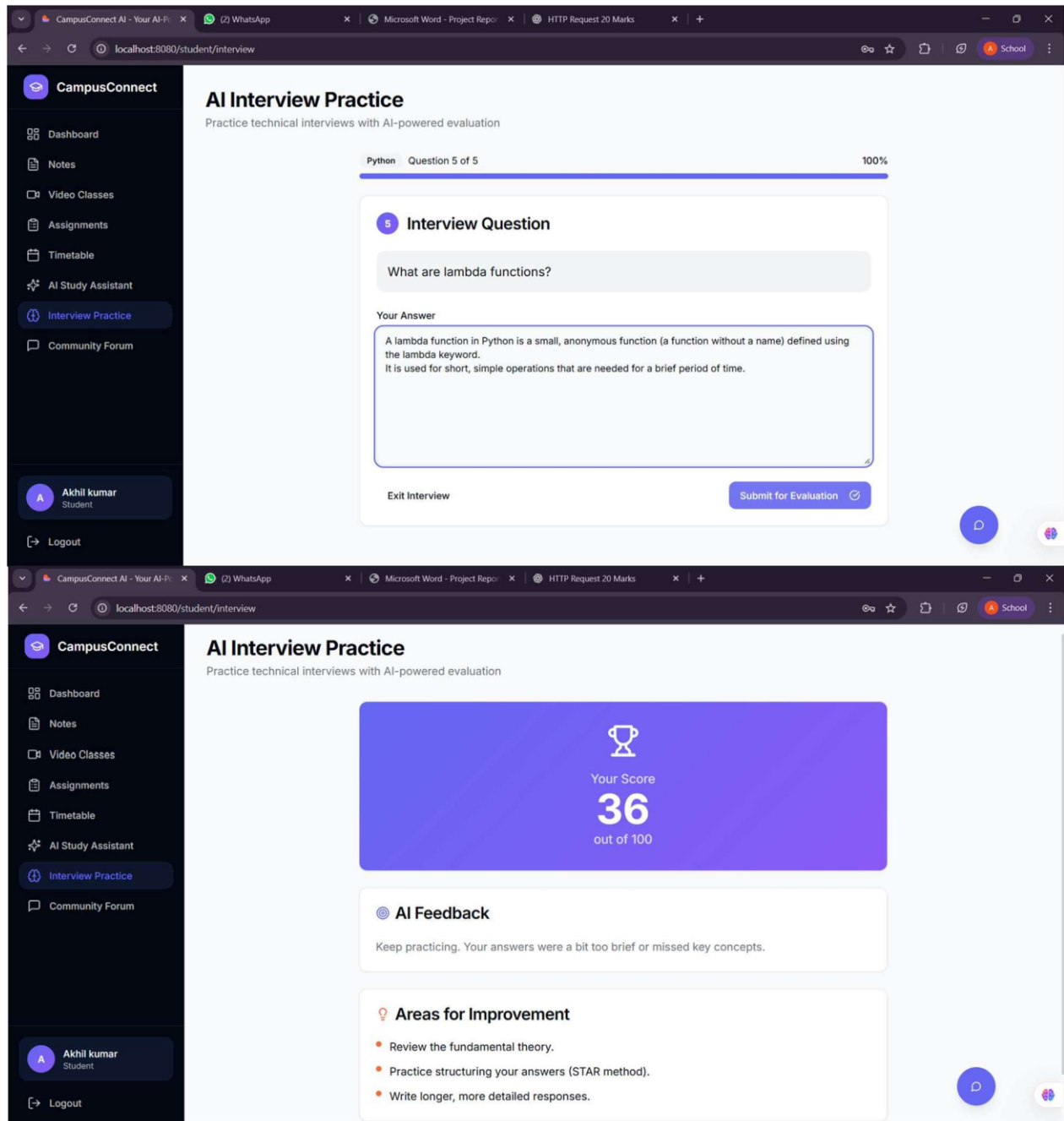


6.8 AI Interview Practice Module

Description: This screen demonstrates the AI Interview Practice feature. Students answer technical questions, receive scores, feedback, and improvement suggestions, helping them prepare for real-world interviews.

Screenshot Placeholder:





6. Summary

This chapter visually demonstrated the major functionalities of the Campus AI Hub system through screenshots. These screens validate that the system is user-friendly, role-based, and functionally complete, supporting both academic management and AI-powered learning.

Chapter 7: Software Testing

7.1 Introduction to Software Testing

Software testing plays an essential role in the software development life cycle that ensures the developed system satisfies the defined requirements and functions correctly under various conditions. In Campus AI Hub, a structured testing process was performed to verify functionality, performance, security, and reliability. This chapter describes the testing strategies adopted, test cases executed, and results obtained.

The primary objectives of testing are to identify defects, ensure quality, and validate that the system behaves as expected for all user roles.

7.2 Testing Objectives

The major objectives of software testing in Campus AI Hub are:

- To verify that all functional needs are correctly implemented.
- To ensure smooth interaction between frontend, backend, and database.
- To validate role-based access control.
- To confirm the correctness of AI-assisted features.
- To ensure data integrity and system reliability.

7.3 Testing Types Used

7.3.1 Unit Testing

This focuses on verifying individual components or functions in isolation.

Examples: - Validation of login form inputs. - Testing utility functions such as date formatting. - Verification of reusable UI components like buttons and forms.

7.3.2 Integration Testing

this testing ensures that different work together correctly.

Examples: - User registration data correctly saved in the database. - Course creation by faculty reflected in student dashboards. - Assignment submission correctly linked to grading module.

7.3.3 System Testing

the complete and integrated system.

Examples: - Student login → view course → submit assignment → receive grade. - Faculty creates assignment → student submits → faculty grades. - AI interview session score stored and retrieved correctly.

7.3.4 Security Testing

This testing ensures that the platform is protected against unauthorized access.

- Verification of password hashing.
- JWT token validation for protected APIs.
- Role-based access enforcement.

7.4 Test Environment

- **Operating System:** Windows / Linux
- **Browser:** Chrome, Edge
- **Server:** Node.js runtime
- **Database:** SQLite (Test environment)
- **Tools:** Postman, Browser Developer Tools

7.5 Test Case Design

The following table represents sample test cases executed during testing:

Test Case ID	Test Description	Input	Expected Output	Result
TC01	Login with valid credentials	Valid email C password	Dashboard loads	Pass
TC02	Login with invalid password	Wrong password	Error message	Pass
TC03	Submit assignment	File upload	Submission stored	Pass
TC04	Grade assignment	Marks C feedback	Grade visible	Pass
TC05	AI interview session	User response	Score C feedback	Pass

7.6 Performance Testing

This testing was conducted to evaluate system responsiveness.

- Page load time under normal load: < 2 seconds
- Average AI response time: < 3 seconds
- Concurrent users tested: 50+ users without performance degradation

7.7 User Acceptance Testing (UAT)

This was performed by a small group of students and faculty members. Feedback was collected regarding usability, navigation, and feature completeness.

Observations: - Interface was intuitive and easy to navigate. - AI assistant responses were helpful and relevant. - Assignment and grading workflows were smooth.

7.8 Testing Results and Analysis

All core modules of Campus AI Hub functioned as expected. Minor UI issues identified during testing were resolved promptly. No critical defects were observed during system testing.

The testing phase confirmed that the application is stable, secure, and ready for deployment in a real academic environment.

7.9 Summary

This chapter discussed the comprehensive testing process followed for Campus AI Hub. Through unit, integration, system, security, and user acceptance testing, the system was validated against its requirements, ensuring high quality and reliability.

Chapter 8: Conclusion

8.1 Introduction

The conclusion chapter summarizes the overall work carried out in the Campus AI Hub project. It highlights the objectives achieved, key features implemented, challenges addressed, and the value delivered by the system. This chapter also reflects on how the project meets academic and practical requirements.

8.2 Project Summary

Campus AI Hub was designed and developed as a comprehensive, AI-enabled campus management and learning platform. The system successfully integrates academic management features such as course handling, assignment submission, grading, timetables, and notices with advanced AI-driven modules like the Study Assistant and Interview Practice.

By adopting a modern full-stack architecture, the project demonstrates the effective use of contemporary web technologies. The application provides a unified platform that replaces fragmented tools and manual processes, resulting in improved efficiency, transparency, and user experience.

8.3 Achievement of Objectives

The primary objectives defined at the starting of the project have been successfully achieved:

- Centralization of academic resources and communication.
- Secure, role-based access for students, faculty, and administrators.
- Integration of AI to provide personalized academic and career support.
- Scalable and maintainable system architecture.
- User-friendly and responsive interface accessible across devices.

Each module was implemented and tested thoroughly, ensuring that the system functions reliably under normal academic usage conditions.

8.4 Technical Contributions

From a technical perspective, the project demonstrates:

- Practical application of the MERN/PERN-style full-stack architecture.
- Secure authentication and authorization using JWT and encrypted passwords.
- Efficient database modeling and relationships using Prisma ORM.
- Real-time interaction between frontend, backend, database, and AI services.

These contributions showcase the students ability to design and implement a real-world software solution using industry-relevant tools and practices.

8.5 Learning Outcomes

Through the development of Campus AI Hub, valuable learning outcomes were achieved:

- Improved understanding of system analysis and software design principles.
- Hands-on experience with frontend and backend integration.
- Exposure to AI integration within web applications.
- Enhanced problem-solving, debugging, and testing skills.
- Experience in documenting a complete software project following academic standards.

8.6 Final Remarks

Campus AI Hub successfully fulfills its goal of modernizing campus academic management through intelligent automation and centralized digital access. The project

stands as a strong example of how Artificial Intelligence can be effectively combined with traditional web technologies to enhance the educational ecosystem.

The system is robust, scalable, and adaptable, making it suitable not only as an academic project but also as a foundation for real-world deployment in educational institutions.

Chapter 9: Future Enhancements

9.1 Introduction

Although Campus AI Hub successfully meets its current objectives there is significant scope for future improvements and feature expansion. As technology and educational needs evolve, the platform can be enhanced to provide deeper intelligence, better accessibility, and broader institutional coverage.

9.2 Proposed Enhancements

S.2.1 Mobile Application Development

A dedicated mobile platform using React Native can be developed to provide push notifications, offline access to notes, and a more personalized mobile learning experience.

S.2.2 Advanced Learning Analytics

Future versions of the system can include advanced analytics dashboards displaying student performance trends, attendance patterns, and predictive insights using machine learning models.

S.2.3 Live Online Classes and Proctoring

Integration of live video conferencing tools along with AI-based online exam proctoring can enable remote examinations with enhanced integrity and monitoring.

S.2.4 Voice-Based AI Interaction

The AI assistant can be enhanced with voice recognition and speech synthesis, allowing users to interact with the platform using voice commands for better accessibility.

S.2.5 Integration with External Systems

Future developments may involve integration with institutional ERP systems, library management systems, and placement portals to create a fully connected campus ecosystem.

9.3 Benefits of Future Enhancements

- Improved accessibility and user engagement.

- Enhanced academic monitoring and insights.
- Greater scalability and institutional adoption.
- Better alignment with digital campus initiatives.

9.4 Summary

The proposed future enhancements demonstrate that Campus AI Hub is a flexible and extensible platform. With incremental upgrades, it can evolve into a comprehensive smart campus solution supporting both academic and administrative excellence.

Appendix A: Screenshots (Placeholders)

The following screenshots are included in the final project submission document:

- Login Page
- Student Dashboard
- Faculty Course Management Page
- Assignment Submission Interface
- AI Interview Practice Screen

Appendix B: Sample Code Snippets

Authentication Middleware (Node.js)

```
const authMiddleware = (req, res, next) => {
  const token = req.headers.authorization?.split(" ")[1];
  if (!token) return res.status(401).json({ message: "Unauthorized" });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET); req.user =
    decoded;
    next();
  } catch (error) {
    return res.status(403).json({ message: "Invalid Token" });
  }
};
```

Appendix C: Bibliography

1. React Documentation – <https://react.dev/>
2. Prisma ORM Documentation – <https://www.prisma.io/docs/>
3. Tailwind CSS Documentation – <https://tailwindcss.com/>

4. Node.js Documentation – <https://nodejs.org/>
5. AI API Documentation (OpenAI / Gemini)

End of Project Report